

具身智能平台部署与运维

ROS2 + Gazebo + Docker 全栈仿真体系

2026 金砖赛 · 人工智能应用创新赛项

目录

#	主题	内容
1~3	理论基础	具身智能定义、ROS2、Gazebo 仿真
4~7	基础设施	Docker、Compose、code-server、rosbridge
8~11	应用层	导航控制、OpenClaw、Dify、Linux 部署
12~15	项目实操	环境搭建、Docker 操作、部署验证、故障排查

01 · 什么是具身智能

具身智能 (Embodied AI, EAI)

“ 智能体必须具备物理或仿真的“身体”，通过与环境的实时交互来感知、推理和行动 ”

核心闭环

感知 (Perception) → 决策 (Planning) → 行动 (Action) ↻

技术交叉领域

机器人学 + 深度学习 + LLM 智能体 + 云原生容器化 + 边缘计算

具身智能五层架构



本模块构建的 "仓库小车仿真平台" 完整覆盖此五层体系。

具身智能核心挑战

挑战	本模块应对策略
仿真与现实差距 (Sim-to-Real Gap)	Gazebo 高保真物理引擎
实时性要求	ROS2 实时通信框架
多组件协同	Docker Compose 统一编排
环境一致性	Docker 容器化，一次构建到处运行
人机交互	OpenClaw 智能体网关 + LLM

发展历程

1990s 传统机器人期
硬编码 PID 控制

→

2015 深度学习融合期
端到端 DRL

→

2022 大模型智能体期
LLM + 机器人自然语言控车

02 · ROS2 机器人操作系统

“

ROS2 是机器人世界的**"安卓系统"**——标准 API + 通信机制，模块即插即用

”

ROS1 vs ROS2

特性	ROS1	ROS2
通信协议	自定义 TCP/UDP	DDS （工业级分布式标准）
操作系统	仅 Ubuntu	Ubuntu / Windows / macOS
实时性	不支持	✓ 实时控制
多机器人	需额外配置	✓ 原生支持
本模块使用	✗	✓ ROS2 Humble

ROS2 核心概念

节点 (Node) + 话题 (Topic) = 发布/订阅模式

ROS2 话题通信 — 发布/订阅模式



本模块关键话题

话题名称	消息类型	说明
/bcr_bot/cmd_vel	geometry_msgs/Twist	速度控制指令：线速度 + 角速度
/bcr_bot/scan	sensor_msgs/LaserScan	激光雷达数据：障碍物距离信息
/bcr_bot/odom	nav_msgs/Odometry	里程计：机器人位置和速度估计

底层通信： DDS (Data Distribution Service) — 自动发现 · QoS 策略 · 实时传输 · 去中心化
ROS2 使用 DDS 替代 ROS1 的自定义协议，支持实时控制、多机器人、跨平台

ROS2 关键话题与 DDS

本模块三大话题

话题名称	消息类型	说明
/bcr_bot/cmd_vel	Twist	速度控制指令
/bcr_bot/scan	LaserScan	激光雷达数据
/bcr_bot/odom	Odometry	里程计位置估计

DDS 通信协议特性

特性	说明
自动发现	节点启动自动发现，无需手动配置 IP
QoS 策略	可配置可靠性、持久性、截止时间
去中心化	无 Master 节点，天然支持分布式
实时性	支持 RTPS 协议

ROS2 常用命令

```
# 查看所有运行中的节点
ros2 node list

# 查看所有话题
ros2 topic list

# 监听话题数据
ros2 topic echo /bcr_bot/cmd_vel

# 发布速度指令
ros2 topic pub /bcr_bot/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.5}, angular: {z: 0.0}}"

# 查看话题详情
ros2 topic info /bcr_bot/scan

# 查看节点信息
ros2 node info /bcr_bot_controller
```

03 · Gazebo 机器人仿真

“ Gazebo 是机器人的**"虚拟训练场"**——无真实硬件即可模拟物理世界行为 ”

核心能力



Gazebo 仓库仿真场景

本模块使用 Gazebo 模拟了一个完整的仓库物流场景：

仿真元素	说明
仓库环境	墙壁、货架、障碍物
差分驱动机器人	两轮驱动 + 万向轮
激光雷达 (LiDAR)	360° 扫描检测障碍物
相机	RGB 摄像头视觉感知
物理引擎	重力、碰撞、摩擦力

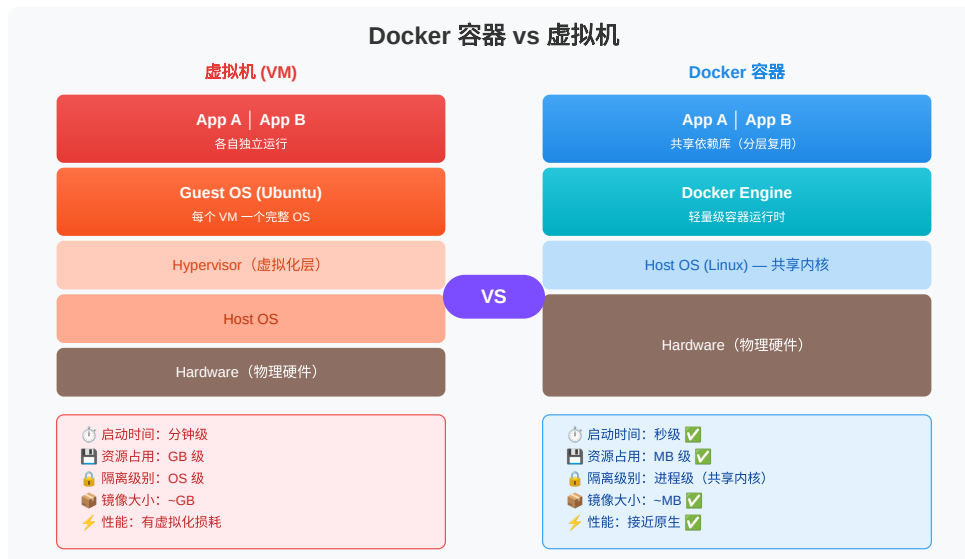
URDF / SDF 模型格式

格式	用途
URDF	描述机器人结构（关节、连杆）
SDF	描述仿真世界（光照、物理属性）

04 · Docker 容器技术

“ Docker 像**"集装箱"**——软件+依赖装进标准化箱子，到处可用 ”

容器 vs 虚拟机



Docker 核心三要素

要素	说明	类比
镜像 (Image)	只读模板，分层存储	模具
容器 (Container)	镜像的运行实例 + 可读写层	铸件
仓库 (Registry)	镜像存储分发中心	仓库

镜像分层结构

应用层 → 中间件层 (ROS2) → 基础层 (Ubuntu 22.04)
每层只读，共享复用 → 节省磁盘空间

本模块端口映射

宿主机	容器	服务
18080	18080	code-server 浏览器 IDE
8888	8888	仿真 Web 仪表盘
28789	28789	OpenClaw 智能体面板
19090	19090	rosbridge WebSocket

Docker 常用命令

— 镜像操作 —

`docker images`

`docker load -i <file>.tar`

`docker save -o <file>.tar <image>`

列出本地镜像

离线导入镜像

导出镜像为 tar

— 容器操作 —

`docker ps`

`docker run -d --name <name> <image>`

`docker exec -it <container> bash`

`docker logs -f <container>`

`docker stop <container>`

`docker rm <container>`

运行中的容器

后台启动容器

进入容器终端

实时查看日志

停止容器

删除容器

— 数据持久化 —

`-v` 宿主机路径:容器路径

Bind Mount 挂载

05 · Docker Compose 多容器编排

通过 YAML 文件一键启动整个应用栈

Docker Compose 多容器编排架构



Docker Compose 配置与命令

`docker-compose.yml` 核心结构

```
services:
  vehicle-dev:                # 主开发容器
    image: ydy-eia/vehicle-agent:latest
    ports:
      - "18080:18080"         # code-server
      - "19090:19090"         # rosbridge
      - "28789:28789"         # OpenClaw
    volumes:
      - ../robot-code:/workspace
    network_mode: host

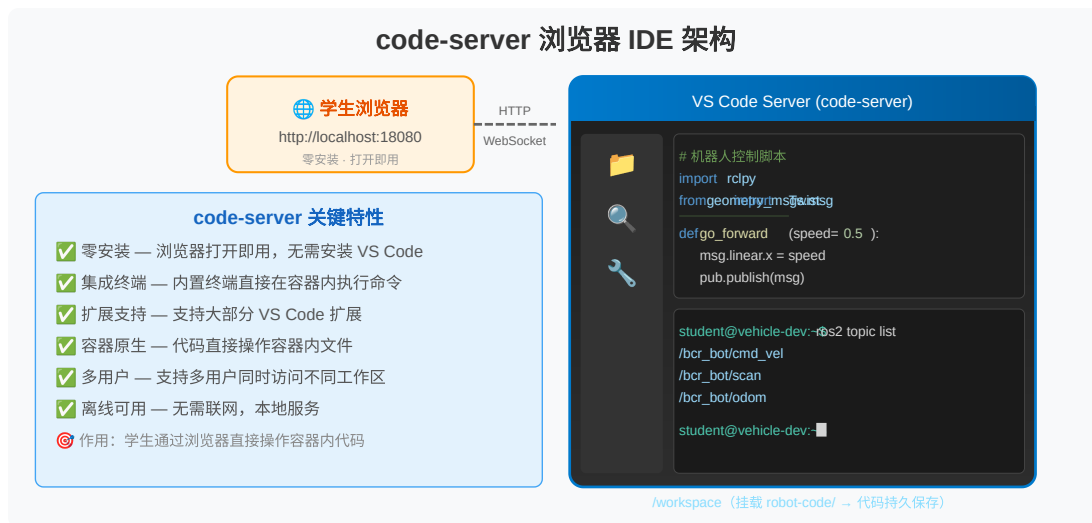
  sim-web:                    # 仿真 Web 容器
    image: ydy-eia/sim-web:latest
    ports:
      - "8888:8888"
```

常用命令

<code>docker compose up -d</code>	# 后台启动所有服务
<code>docker compose down</code>	# 停止并移除
<code>docker compose ps</code>	# 查看状态
<code>docker compose logs -f <service></code>	# 查看日志

06 · code-server 浏览器 IDE

code-server = VS Code 的 Web 版，浏览器即 IDE

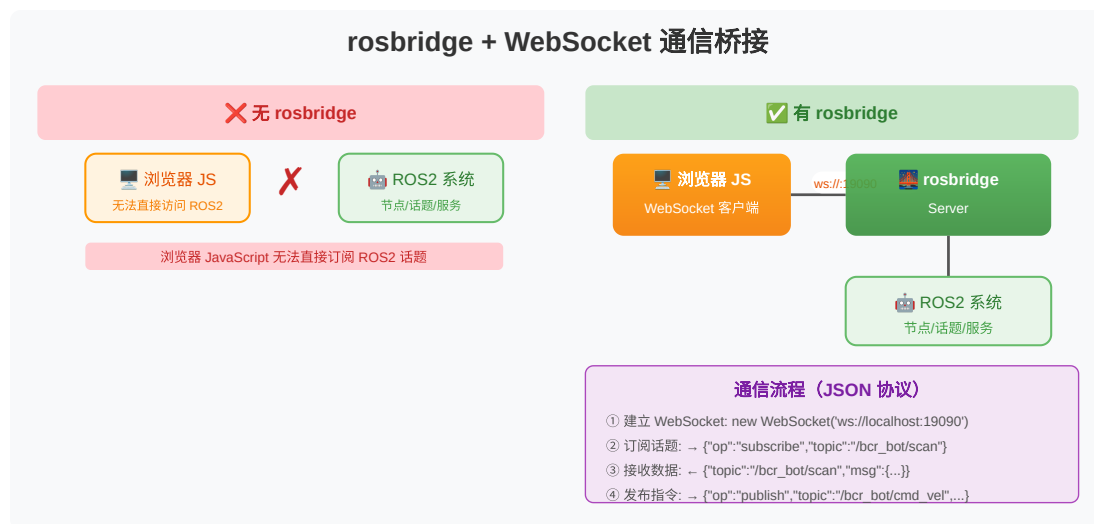


关键特性

特性	说明
零安装	浏览器打开即用
集成终端	直接在容器内执行命令
扩展支持	Python、Docker 等 VS Code 扩展
容器原生	代码直接在容器文件系统操作

07 · rosbridge 与 WebSocket

“ rosbridge = ROS2 话题系统 ↔ 浏览器 JavaScript 的通信桥梁 ”



robridge 通信流程

1. JS 建立 WebSocket: `new WebSocket('ws://localhost:19090')`
2. 订阅 ROS2 话题:
→ `{"op": "subscribe", "topic": "/bcr_bot/scan"}`
3. 接收传感器数据 (JSON):
← `{"topic": "/bcr_bot/scan", "msg": {...}}`
4. 发布控制指令:
→ `{"op": "publish", "topic": "/bcr_bot/cmd_vel", "msg": {...}}`

WebSocket vs HTTP

特性	HTTP	WebSocket
通信模式	请求-响应（单向）	全双工（双向）
连接方式	短连接	长连接
实时性	需轮询	实时推送
本模块用途	code-server 页面	仿真数据实时推送

08 · 机器人导航与控制

差分驱动模型

本模块仓库小车使用**差分驱动**——两轮速度差实现转向：

“

前进： $v_{\text{left}} = v_{\text{right}}$ | 原地旋转： $v_{\text{left}} = -v_{\text{right}}$

”

Twist 消息与 LaserScan

Twist 速度控制

linear.x → 前进/后退 (m/s)	angular.z → 绕 Z 轴旋转 (rad/s)
正值=前进	正值=左转(逆时针)
负值=后退	负值=右转(顺时针)

控制命令示例

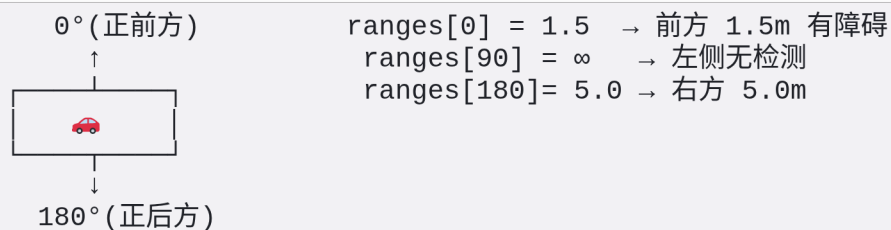
```
# 前进 0.5 m/s
ros2 topic pub /bcr_bot/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.5}, angular: {z: 0.0}}"

# 原地旋转 1.0 rad/s
ros2 topic pub /bcr_bot/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.0}, angular: {z: 1.0}}"

# 停止
ros2 topic pub /bcr_bot/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.0}, angular: {z: 0.0}}"
```

LaserScan 解读与避障

激光雷达数据

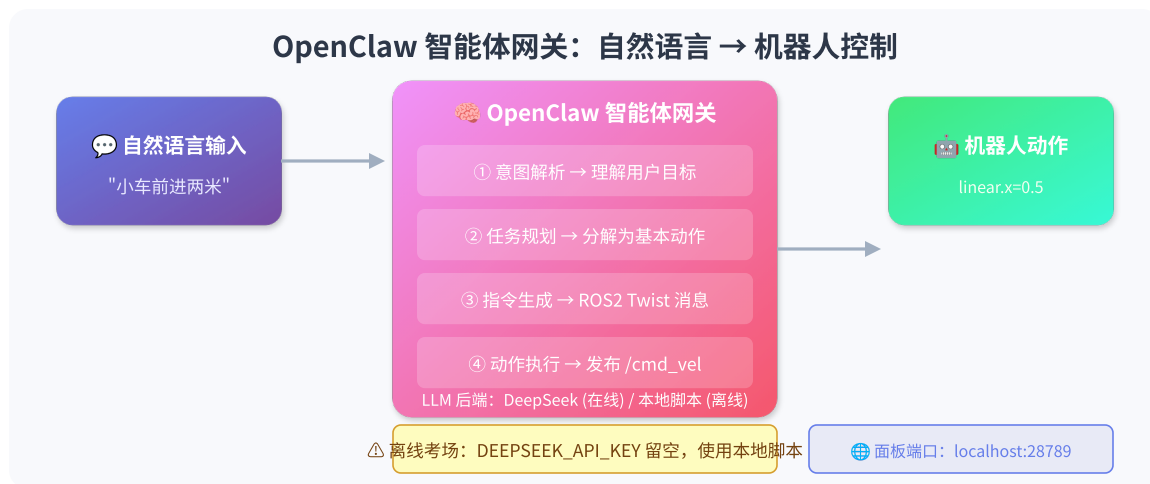


简单避障逻辑

```
def avoid_obstacle(scan_data):  
    front = scan_data.ranges[0]  
    left = scan_data.ranges[90]  
    right = scan_data.ranges[270]  
    if front < 0.5:  
        turn_right() if left > right else turn_left()  
    else:  
        go_forward()
```

09 · OpenClaw 智能体网关

“ OpenClaw = LLM 与机器人之间的**"翻译层"**, 自然语言 → 机器人指令 ”



离线 vs 在线

模式	说明	适用
在线模式	调用云端 LLM API (DeepSeek)	需联网 + API Key
离线模式	本地脚本执行预定义逻辑	✅ 考场默认

10 · Dify LLM 应用平台

“ 可视化 Prompt 编排 + RAG + Agent，非程序员也能快速构建 AI 应用 ”



在本模块中的角色

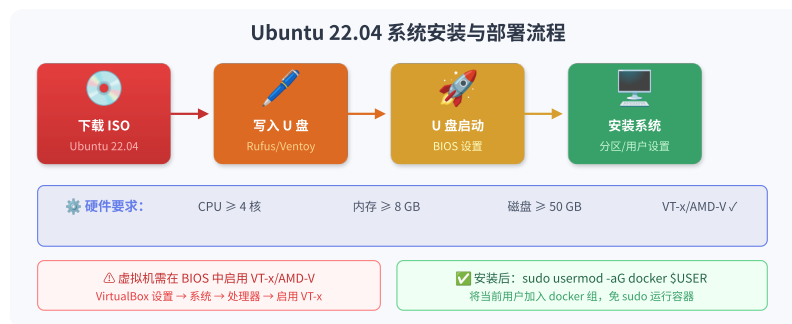
- 可选 LLM 编排层，构建复杂自然语言控车逻辑
- 支持离线部署：`docker save/load` 导入镜像包
- 离线考场无需 Dify，自然语言控车通过 `drive.sh` 等本地脚本完成

11 · Linux 系统部署基础

为什么选 Ubuntu 22.04

原因	说明
ROS2 Humble Tier-1	官方首选支持平台
LTS 至 2027	长期稳定性保障
Docker 完整支持	官方提供完整支持
内核 5.15	虚拟化和容器原生支持良好

U 盘启动安装流程



虚拟化要求

BIOS/UEFI 中需启用:

VT-x (Intel) / AMD-V (AMD)

→ 虚拟机中运行 Docker 时必需

← CPU 虚拟化扩展

Linux 常用命令速查

```
# — 系统信息 —
lsb_release -a          # Ubuntu 版本
uname -r                # 内核版本
df -h / free -h         # 磁盘 / 内存

# — 用户与权限 —
sudo <command>          # 管理员权限
usermod -aG docker $USER # 加入 docker 组
groups $USER             # 查看用户组

# — 文件操作 —
mkdir -p /path/to/dir   # 递归创建目录
cp -r src dst           # 递归复制
chmod +x script.sh      # 添加执行权限

# — 进程与端口 —
ps aux                  # 所有进程
lsof -i :8888           # 查看端口占用
```

12 · 项目实操：环境搭建

硬件要求

项目	最低建议
CPU	4 核及以上
内存	16 GB (8 GB 可勉强运行)
磁盘	60 GB 可用空间
系统	Ubuntu 22.04 Desktop 64 位
虚拟化	已启用 VT-x / AMD-V

安装 Docker

```
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli \
    containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker "$USER"      # 重新登录生效
docker --version                      # 验证
```

13 · 项目实操：导入镜像与部署

导入离线镜像

```
cd ~/exam/module_c_eai_platform/images
docker load -i ydy-eia-vehicle-agent.tar
docker load -i ydy-eia-sim-web.tar
docker images | grep ydy-eia      # 验证
```

配置并启动

```
cd ~/exam/jushenai-vehicle-agent
cp docker/.env.example docker/.env    # 配置环境变量
cd docker && ./up.sh                  # 一键启动
```

端口一览

端口	服务
localhost:18080	code-server 浏览器 IDE
localhost:8888	仿真 Web 3D 仪表盘
localhost:28789	OpenClaw 智能体面板
localhost:19090	rosbridge WebSocket

14 · 项目实操：验证清单

按顺序自检，全部通过即可开始答题：

- [] `docker ps` 能看到 `ydy-eia-vehicle-dev`、`ydy-eia-sim-web` 两个容器
- [] 浏览器打开 <http://localhost:8888> → 3D 仓库场景
- [] 浏览器打开 <http://localhost:18080> → code-server 工作区 `/workspace`
- [] 在 code-server 终端执行：

```
cd /workspace  
./scripts/drive.sh "小车前进"
```

仿真中小车应前进数秒后停止。

⚠ 容器启动后约 **90 秒** 仿真才完全就绪

目录挂载关系

宿主机路径	容器内路径	用途
robot-code/	/workspace	学生代码工作区（code-server 默认打开）
项目根目录	/project	平台脚本、仿真场景资源

“

/workspace 下修改的代码**直接保存到宿主机**，重启容器不丢失

”

常用操作

```
./show-urls.sh    # 查看服务地址
./down.sh         # 停止环境
./up.sh           # 重新启动
docker logs -f ydy-eia-vehicle-dev    # 查看仿真日志
```

15 · 故障排查

现象	解决办法
<code>docker load</code> 空间不足	<code>docker system prune -a</code> 清理
code-server 打不开	使用 http 而非 https；检查端口
仿真 Web 无画面	等待 90 秒； <code>docker logs</code> 检查
<code>drive.sh</code> 无反应	确认在容器内终端执行；确认仿真就绪
端口冲突	修改 <code>docker/.env</code> 端口后重新启动
权限 denied (docker)	<code>sudo usermod -aG docker \$USER</code> 后重新登录

容器内检查 ROS 话题

```
docker exec -u dev ydy-eia-vehicle-dev bash -lc \  
'source /opt/ros/humble/setup.bash && ros2 topic list'
```

应包含 `/bcr_bot/cmd_vel`、`/bcr_bot/scan` 等。

课程总结

你已掌握的技能

环节	技能点
 体系理解	具身智能定义、五层架构、发展历程
 ROS2	节点/话题/服务/DDS 通信机制
 仿真	Gazebo 物理仿真、URDF/SDF 模型
 容器化	Docker 镜像/容器、Docker Compose 编排
 开发环境	code-server + rosbridge + WebSocket
 控制	差分驱动、Twist、LaserScan 避障
 智能体	OpenClaw 网关、Dify 平台
 部署运维	Linux 安装、离线部署、故障排查

关键技术路径

仓库小车仿真平台的完整技术栈

```
Ubuntu 22.04 (宿主机)
├── Docker Engine
│   ├── vehicle-dev 容器 (ROS2 + Gazebo + code-server + OpenClaw)
│   │   └── rosbridge ↔ 浏览器 WebSocket
│   └── sim-web 容器 (3D 仿真 Web 仪表盘)
```

核心数据流

```
浏览器 IDE → code-server → 编辑代码 → drive.sh
├── ros2 topic pub /cmd_vel → Gazebo 仿真小车运动
└── /scan 激光雷达数据 → rosbridge → 浏览器实时显示
```

参考资料

- **ROS 2 Documentation:** <https://docs.ros.org/en/humble/>
- **Gazebo Sim:** <https://gazebo.org/>
- **Docker Docs:** <https://docs.docker.com/>
- **code-server:** <https://github.com/coder/code-server>
- **rosbridge_suite:**
https://github.com/RobotWebTools/rosbridge_suite
- **OpenClaw:** <https://github.com/openclaw>
- **Dify:** <https://dify.ai/>

Thank You

准备好了吗？开始搭建你的具身智能平台！

```
docker compose up -d
```

“

 配套文档：

- 基础知识手册
- 实操手册
- 技术架构说明

”